

University Name: AL-Razi University

Project Title : Task Management REST Service

Course Name: Cloud Computing

Semester: First Semester 2026/2027
Submission Date: September 21, 2026

Team Members:

- [15] - عبدالله نبيل محمد اليافعي
- [1] - أسامة علي محمد الجهوري
- [25] - معتز ناصر زيد ابكر
- [18] - عمر بدر الدين عمر الاسيدي

Instructor:

Theoretical Instructor Name: مهند المشرقي

Practical Instructor Name: مجد السريحي

جدول المحتويات

1. الملخص التنفيذي
2. معمارية النظام
3. تحليل: البيئات الافتراضية مقابل الحاويات
4. تحليل تحسين طبقات ملف Docker
5. الممارسات الأمنية
6. معمارية الشبكة
7. معمارية التخزين
8. التحقق من استمرارية البيانات
9. سيناريوهات الفشل والتعافي
10. سجل الحاويات وإدارة النسخ
11. أدلة الاختبار
12. مساهمات الفريق

1. الملخص التنفيذي (Executive Summary)

يُقدِّم هذا التقرير التقني وصفاً شاملاً لمشروع خدمة إدارة المهام (Task Management REST Service)، وهو نظام RESTful API مُصمَّم لإجراء عمليات CRUD الكاملة (الإنشاء، والقراءة، والتحديث، والحذف) على موارد المهام. يهدف المشروع إلى تطبيق المفاهيم الأساسية لتقنية الحاويات (Containerization) وتنسيق الخدمات المتعددة (Multi-Service Orchestration) في بيئة إنتاجية مُحاكاة.

تم بناء الطبقة الخلفية (Backend) باستخدام إطار العمل FastAPI المبني على لغة Python 3.13، والذي يتميز بأدائه العالي ودعمه المُدمَج للتوثيق التلقائي عبر معيار OpenAPI. تُخزَّن البيانات في قاعدة بيانات PostgreSQL 17 المعروفة بموثوقيتها وتوافقها الكامل مع معايير ACID. يعمل خادم Nginx بوصفه وكيلًا عكسيًا (Reverse Proxy) ونقطة دخول وحيدة (Single Entry Point) للنظام، مما يُوفِّر طبقة عزل أمنية إضافية بين العميل الخارجي والخدمات الداخلية.

تم تحويل جميع مكونات النظام إلى حاويات Docker مستقلة، وتُدار بشكل مُنسَق عبر Docker Compose v2. يشمل النظام ثلاث خدمات رئيسية: nginx، وapi، وdatabase، وجميعها تتواصل عبر شبكة bridge داخلية معزولة (app-network). تم ضمان استمرارية البيانات (Data Persistence) من خلال استخدام Named Volume مُخصَّص لقاعدة البيانات. كما تم رفع الصورة النهائية (Docker Image) إلى سجل Docker Hub تحت المستودع osama2sv/task-management-api بنظام إصدارات مُحكَم (latest v1.0.0).

يشمل هذا التقرير تحليلاً مُعمَّقاً لمعمارية النظام، وتحسين طبقات Dockerfile، والممارسات الأمنية المُطبَّقة، ومعمارية الشبكة والتخزين، وسيناريوهات الفشل والتعافي، بالإضافة إلى أدلة اختبار شاملة تغطي جميع واجهات API وحالات الخطأ.

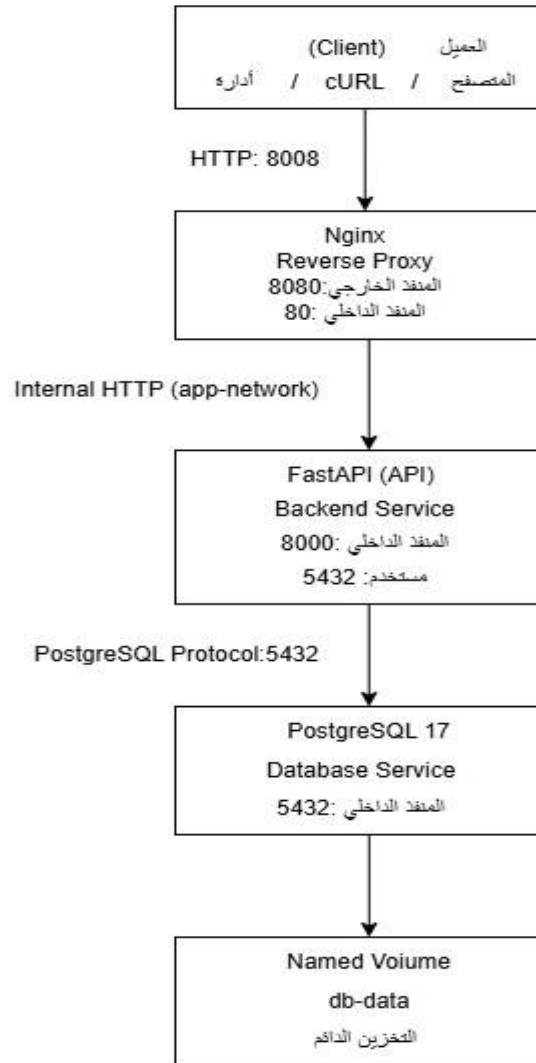
المكوّن	التقنية المُستخدمة
لغة البرمجة	Python 3.13
إطار العمل الخلفي	FastAPI
خادم التطبيق	Uvicorn
قاعدة البيانات	PostgreSQL 17-alpine
الوكيل العكسي	Nginx
تقنية الحاويات	Docker
تنسيق الخدمات	Docker Compose v2

المكوّن	التقنية المستخدمة
اختبار الواجهات	cURL
سجل الصور	Docker Hub

2. معمارية النظام (System Architecture)

2.1 نظرة عامة على المعمارية

يتبنى النظام نمط المعمارية متعددة الطبقات (Multi-Tier Architecture) ، حيث تُفصل المسؤوليات بين ثلاث خدمات مُحثّوة (Containerized Services) تعمل بتناغم داخل بيئة Docker Compose :



2.2 وصف المكونات

الطبقة الأولى — Nginx (الوكيل العكسي): يعمل Nginx كنقطة الدخول الوحيدة للنظام بأكمله. يستقبل جميع الطلبات الخارجية الواردة على المنفذ 8080 ويوجهها داخلياً إلى خدمة الـ API عبر بروتوكول HTTP الداخلي. يُعرّف upstream backend في ملف nginx.conf ليُشير إلى اسم خدمة Docker (api:8000) مما يُمكن Nginx من استخدام آلية DNS الداخلية لـ Docker لاكتشاف الخدمة تلقائياً. كما يُمرّر Nginx ترويسات (Headers) مهمة مثل X-Real-IP و X-Forwarded-For و X-Forwarded-Proto للحفاظ على معلومات العميل الأصلية.

الطبقة الثانية — FastAPI (واجهة برمجة التطبيقات): تُشكّل هذه الطبقة جوهر منطق الأعمال (Business Logic) للنظام. تم بناؤها باستخدام إطار العمل FastAPI وتعمل على خادم Uvicorn (خادم ASGI عالي الأداء). تُوفّر الخدمة خمسة مسارات (Endpoints) رئيسية لإدارة المهام، وتتواصل مع قاعدة البيانات عبر مكتبة psycopg (الإصدار الثالث). يعمل التطبيق داخل الحاوية بصلاحيات مستخدم غير جذري (Non-root User) باسم appuser لتعزيز الأمان.

الطبقة الثالثة — PostgreSQL (قاعدة البيانات): تُستخدم صورة postgres:17-alpine الخفيفة لتشغيل قاعدة البيانات. يتم تهيئة الجدول tasks تلقائياً عند أول تشغيل عبر سكريبت init.sql المثبت في مجلد /docker-entrypoint-initdb.d/. تُخزّن جميع البيانات على Named Volume باسم db-data لضمان استمراريتها خارج دورة حياة الحاوية.

2.3 تفاعل المكونات وتسلسل الطلبات

عند وصول طلب HTTP من العميل، يمر بالتسلسل التالي:

1. يصل الطلب إلى Nginx على المنفذ 8080 من الجهاز المضيف (Host).
2. يُوجّه Nginx الطلب داخلياً إلى FastAPI عبر الشبكة app-network على المنفذ 8000.
3. يُعالج FastAPI الطلب ويتواصل مع PostgreSQL عبر نفس الشبكة على المنفذ 5432.
4. تعود الاستجابة بالتسلسل العكسي حتى تصل إلى العميل.

2.4 أدلة بصرية

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose config --services
database
api
nginx
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose config --volumes
db-data
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose config | Select-String -Pattern
'published|expose'

expose:
expose:
  published: "8080"
```

شرح الصورة 01: توضح هذه الصورة مُخرَج أمر `docker compose config` الذي يعرض التكوين النهائي المُدمَج لجميع خدمات النظام الثلاث (database ، api ، nginx) مع إعدادات الشبكات والتخزين وفقاً لملف `compose.yaml`.

NAME	PORTS	IMAGE	COMMAND	SERVICE	compose ps CREATED	STATUS
task-api	8080/tcp	task-management-api:v1.0.0	"uvicorn api.main:ap..."	api	24 seconds ago	Up 18 seconds (healthy)
task-database	5432/tcp	postgres:17-alpine	"docker-entrypoint.s..."	database	24 seconds ago	Up 23 seconds (healthy)
task-nginx	0.0.0.0:8080->80/tcp, [::]:8080->80/tcp	nginx:latest	"/docker-entrypoint..."	nginx	24 seconds ago	Up 12 seconds

شرح الصورة 02: توضح هذه الصورة مُخرَج أمر `docker compose ps` الذي يعرض حالة تشغيل جميع حاويات النظام (task-database ، task-api ، task-nginx) وحالتها الصحية (healthy) والمنافذ المُعرَّضة لكل منها.

3. تحليل: البيئات الافتراضية مقابل الحاويات

(Virtualization vs. Containerization Analysis)

3.1 مقدمة المقارنة

في سياق نشر التطبيقات الحديثة، يُعد الاختيار بين الأجهزة الافتراضية (Virtual Machines - VMs) وتقنية الحاويات (Containers) قراراً معمارياً جوهرياً يؤثر على الأداء وكفاءة استهلاك الموارد وسرعة النشر. يُقدِّم هذا القسم تحليلاً أكاديمياً وعملياً لكلتا التقنيتين، مع تبرير اختيار Docker لهذا المشروع.

3.2 الأجهزة الافتراضية (Virtual Machines)

تعمل الأجهزة الافتراضية عبر طبقة Hypervisor مثل (VMware ESXi أو Hyper-V أو KVM) التي تُحاكي عتاداً افتراضياً كاملاً. تتطلب كل آلة افتراضية نظام تشغيل ضيف (Guest OS) مستقلاً بالكامل بما يشمله من نواة (Kernel) وبرمجيات نظام ومكتبات. يُوفِّر هذا النهج عزلاً قوياً على مستوى العتاد (Hardware-Level Isolation)، لكنه يأتي بتكلفة عالية من حيث:

- استهلاك الموارد: كل VM تتطلب تخصيص ذاكرة عشوائية (RAM) ومساحة تخزين ووحدات معالجة مركزية (vCPU) لنظام التشغيل الضيف بالكامل، حتى لو كان التطبيق بسيطاً.
- حجم الصورة: يتراوح صور الـ VM عادةً بين عدة جيجابايت إلى عشرات الجيجابايت.

- زمن الإقلاع: يستغرق بدء تشغيل VM من عشرات الثواني إلى عدة دقائق بسبب الحاجة لتحميل نظام التشغيل بالكامل.
- النقل والتوزيع: صعوبة في نقل وتوزيع الـ VMs بين البيئات المختلفة بسبب حجمها الكبير.

3.3 الحاويات (Containers)

تعمل الحاويات على مستوى نظام التشغيل (OS-Level Virtualization) وتتشارك نواة (Kernel) النظام المضيف. تحتوي كل حاوية فقط على التطبيق ومكتباته واعتمادياته المباشرة، دون الحاجة لنظام تشغيل ضيف مستقل. يُوفّر هذا النهج مزايا جوهرية:

- كفاءة الموارد: استهلاك أقل بكثير للذاكرة والتخزين ووحدات المعالجة، مما يسمح بتشغيل عشرات أو مئات الحاويات على نفس الجهاز.
- حجم الصورة: يتراوح صور الحاويات عادةً بين عشرات إلى مئات الميجابايت فقط.
- زمن الإقلاع: تبدأ الحاويات في أجزاء من الثانية لأنها لا تُحمّل نواة نظام تشغيل.
- القابلية للنقل: سهولة النقل بين بيئات التطوير والاختبار والإنتاج عبر سجلات الصور (Container Registries).

3.4 مقارنة تفصيلية

معيار المقارنة	الأجهزة الافتراضية (VMs)	الحاويات (Containers)
مستوى العزل	عزل على مستوى العتاد (Hardware)	عزل على مستوى العمليات (Process)
نظام التشغيل	نظام ضيف كامل لكل VM	تتشارك نواة النظام المضيف
حجم الصورة	عدة جيجابايت (GB)	عشرات-مئات ميجابايت (MB)
زمن الإقلاع	دقائق	أجزاء من الثانية
استهلاك الذاكرة	مرتفع (نظام تشغيل كامل)	منخفض (التطبيق فقط)
الكثافة (عدد النسخ لكل خادم)	منخفضة (عشرات)	عالية (مئات-آلاف)
إدارة الاعتماديات	يدوية داخل كل VM	مُعرّفة في Dockerfile
قابلية النقل	محدودة	عالية جداً
أدوات التنسيق	vCenter, OpenStack	Docker Compose, Kubernetes

3.5 تبرير اختيار Docker لهذا المشروع

تم اختيار تقنية الحاويات عبر Docker لهذا المشروع للأسباب التالية:

1. طبيعة التطبيق: المشروع يتكون من ثلاث خدمات مصغرة (Microservices) خفيفة API — و Reverse Proxy وقاعدة بيانات — لا تتطلب أي منها نظام تشغيل ضيف مستقل. استخدام VMs لهذا الحجم من الخدمات يُعدّ إفراطاً غير مُبرّر في استهلاك الموارد.
2. سرعة دورة التطوير: تسمح الحاويات بإعادة بناء ونشر التغييرات في ثوانٍ، مقارنةً بالدقائق المطلوبة لإعادة تهيئة VM.
3. إعادة الإنتاج (Reproducibility): يضمن ملف Dockerfile أن بيئة التشغيل مُطابقة تماماً في كل مرة، مما يُزيل مشكلة "يعمل على جهازي. (Works on my machine)"
4. سهولة التنسيق: يُمكن Docker Compose من تعريف وتشغيل جميع الخدمات والشبكات والتخزين بأمر واحد (docker compose up -d).
5. التوزيع عبر السجل: إمكانية رفع الصورة إلى Docker Hub وسحبها من أي مكان بأمر واحد.

3.6 متى نستخدم كل تقنية؟

تبقى الأجهزة الافتراضية الخيار الأنسب عندما يكون العزل الأمني القوي على مستوى النواة مطلوباً (مثل بيئات Multi-Tenant الحساسة)، أو عند الحاجة لتشغيل أنظمة تشغيل مختلفة مثل (Linux و Windows) على نفس الخادم الفعلي. أما الحاويات فهي الخيار الأمثل للتطبيقات المبنية على معمارية الخدمات المصغرة، وبيئات CI/CD، والنشر السريع.

4. تحليل تحسين طبقات ملف

Docker (Dockerfile Layer Optimization Analysis)

4.1 هيكلية ملف Dockerfile

تم كتابة ملف Dockerfile الخاص بخدمة الـ Backend وفقاً لأفضل الممارسات المُوصى بها لتحسين حجم الصورة وسرعة عملية البناء. فيما يلي المحتوى الكامل للملف مع تحليل كل طبقة:

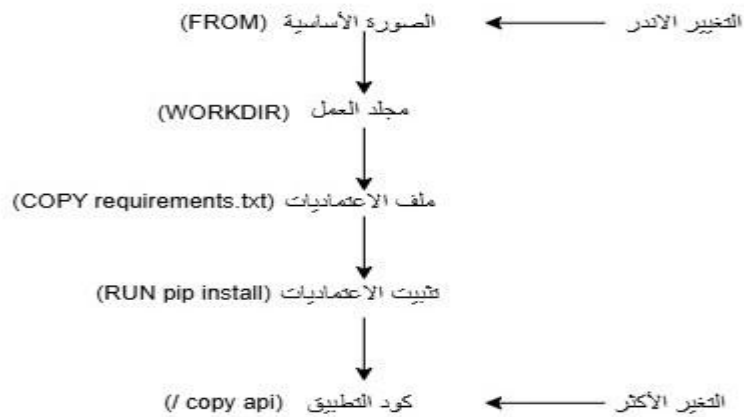
```
FROM python:3.13-slim
WORKDIR /app
COPY api/requirements.txt ./requirements.txt
RUN pip install --no-cache-dir -r requirements.txt
COPY api/ ./api/
RUN groupadd --system appgroup \
    && useradd --system --gid appgroup appuser
USER appuser
EXPOSE 8000
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.2 تحليل كل طبقة (Layer-by-Layer Analysis)

- الطبقة 1 — FROM python:3.13-slim** تم اختيار صورة python:3.13-slim كصورة أساسية (Base Image) بدلاً من الصورة الكاملة python:3.13. تحتوي صورة slim على الحد الأدنى من حزم Debian اللازمة لتشغيل Python، مما يُقلل حجم الصورة بشكل ملحوظ (من حوالي 1 GB في الصورة الكاملة إلى 150 MB في slim). هذا يُقلل أيضاً من مساحة الهجوم الأمني (Attack Surface) بسبب تقليل عدد الحزم المُثبتة.
- الطبقة 2 — WORKDIR /app** تُحدّد مجلد العمل الافتراضي داخل الحاوية. جميع الأوامر اللاحقة (CMD، RUN، COPY) ستُنفَّذ نسبةً إلى هذا المسار. هذه تعليمة بياناتية (Metadata Instruction) لا تُنشئ طبقة ذات حجم ملحوظ.
- الطبقة 3 — COPY api/requirements.txt ./requirements.txt** (نسخ ملف الاعتماديات فقط): تُنسخ قائمة الاعتماديات (Dependencies) وحدها قبل نسخ كود التطبيق. هذا الفصل المتعمّد هو حجر الزاوية في استراتيجية تحسين التخزين المؤقت (Layer Caching Strategy).
- الطبقة 4 — RUN pip install --no-cache-dir -r requirements.txt** (تثبيت الاعتماديات): يتم تثبيت جميع مكتبات Python المطلوبة. استخدام خيار --no-cache-dir يمنع pip من تخزين ملفات التنزيل المؤقتة (Wheel Cache) داخل الصورة، مما يُقلل حجمها النهائي.
- الطبقة 5 — COPY api/ ./api/** (نسخ كود التطبيق): يُنسخ كود المصدر (Source Code) في طبقة منفصلة بعد تثبيت الاعتماديات.
- الطبقة 6 — RUN groupadd ... && useradd ...** (إنشاء مستخدم غير جذري): يتم إنشاء مجموعة نظام (System Group) باسم appgroup ومستخدم نظام (System User) باسم appuser، وذلك لتشغيل التطبيق بصلاحيات محدودة.
- الطبقة 7 — USER appuser** تُبَدّل سياق التنفيذ إلى المستخدم appuser. جميع الأوامر اللاحقة (بما في ذلك CMD) ستعمل بصلاحيات هذا المستخدم.
- الطبقة 8 — EXPOSE 8000** تُوثّق أن الحاوية تستمع على المنفذ 8000. هذه تعليمة توثيقية لا تنشر المنفذ فعلياً.
- الطبقة 9 — CMD [...]** (أمر التشغيل): يُحدّد الأمر الافتراضي لتشغيل خادم Uvicorn مع تطبيق FastAPI.

4.3 استراتيجية التخزين المؤقت للطبقات (Layer Caching Strategy)

تتبع بنية Dockerfile مبدأ "ما يتغير أقل يُوضَع أولاً" (Least Frequently Changed First). الترتيب المُستخدم يُحقّق الفوائد التالية:



السيناريو الأمثل — تغيير كود التطبيق فقط : عندما يُعَدّل المطور كوداً في `main.py` أو `models.py` أو `database.py` دون تغيير الاعتماديات، يستخدم Docker الطبقات المُخزّنة مؤقتاً (Cached Layers) لخطوات `FROM` و `WORKDIR` و `COPY requirements.txt` و `RUN pip install`، ويُعيد بناء طبقة `/COPY api` فقط والطبقات التالية لها. هذا يُقلّص زمن البناء من دقائق (في حال تثبيت الاعتماديات من جديد) إلى ثوانٍ.

السيناريو البديل — تغيير الاعتماديات : إذا تم تعديل ملف `requirements.txt` (مثل إضافة مكتبة جديدة)، تُبطل الطبقة المُخزّنة مؤقتاً لـ `COPY requirements.txt` وجميع الطبقات اللاحقة. هذا السلوك متوقع وضروري لضمان تثبيت الاعتماديات المُحدّثة.

4.4 ملف `dockerignore` ودوره في التحسين

تم إنشاء ملف `dockerignore` لاستبعاد الملفات والمجلدات غير الضرورية من سياق البناء (Build Context) :

```

.venv/
__pycache__ /
*.pyc
*.pyo
*.pyd
.git/
.gitignore
.env
.vscode/
.idea/
*.log
evidence/

```

يُفَلَّل هذا الملف حجم سياق البناء المُرسَل إلى Docker Daemon، ويمنع تسرُّب الملفات الحساسة (مثل env) أو غير الضرورية (مثل venv وevidence) إلى الصورة النهائية.

5. الممارسات الأمنية (Security Practices)

5.1 تشغيل التطبيق بمستخدم غير جذري (Non-root Execution)

تُعَدّ هذه الممارسة من أهم ركائز أمن الحاويات. في ملف Dockerfile، يتم إنشاء مستخدم نظام مُخصَّص وتبديل سياق التنفيذ إليه:

```
RUN groupadd --system appgroup \  
&& useradd --system --gid appgroup appuser  
  
USER appuser
```

هذا يعني أنه حتى في حالة استغلال ثغرة أمنية في التطبيق، فإن المهاجم لن يحصل على صلاحيات root داخل الحاوية، مما يُحدِّد بشكل كبير من نطاق الضرر المُحتَمَل.

5.2 عزل المنافذ (Port Isolation)

تم تصميم النظام بحيث يكون المنفذ الوحيد المنشور على الجهاز المُضيف (Host) هو المنفذ 8080 الخاص بـ Nginx:

```
#خدمة Nginx — المنفذ الوحيد المنشور خارجياً  
ports:  
- "8080:80"  
  
#خدمة API — منفذ داخلي فقط  
expose:  
- "8000"  
  
#خدمة قاعدة البيانات — منفذ داخلي فقط  
expose:  
- "5432"
```

استخدام تعليمة expose بدلاً من ports في خدمتي api و database يعني أن هذه المنافذ مُتاحة فقط داخل شبكة Docker (app-network) ولا يمكن الوصول إليها مباشرةً من الجهاز المُضيف أو من الشبكة الخارجية. هذا يُوفِّر طبقة حماية إضافية تمنع الوصول المباشر غير المُصرَّح به إلى قاعدة البيانات أو خدمة الـ API.

5.3 إخفاء المتغيرات البيئية (env.)

تُخزن بيانات الاعتماد الحساسة (Credentials) مثل اسم قاعدة البيانات واسم المستخدم وكلمة المرور في ملف .env الذي لا يُرفع إلى مستودع الكود (Git Repository):

```
# (القيم الافتراضية للتطوير) .env.example
POSTGRES_DB=taskdb
POSTGRES_USER=taskuser
POSTGRES_PASSWORD=change-me
```

تم إضافة .env إلى كِلِّ من gitignore و dockerignore لضمان:

- **عدم رفعه إلى Git:** حماية بيانات الاعتماد من التسرب في مستودع الكود.
 - **عدم تضمينه في صورة Docker:** منع تسرب البيانات الحساسة إلى سياق البناء أو الصورة النهائية.
- يُقرأ ملف .env تلقائياً بواسطة Docker Compose عند التشغيل، ويتم حقن قيمه كمتغيرات بيئية في الحاويات ذات الصلة عبر آلية {VARIABLE_NAME}\$ في ملف compose.yaml.

5.4 تأمين الوكيل العكسي (Reverse Proxy)

يعمل Nginx كبوابة أمنية (Security Gateway) من خلال:

- **إخفاء البنية الداخلية:** لا يظهر للعميل الخارجي أي تفاصيل عن المنافذ الداخلية أو أسماء الخدمات.
- **تمرير ترويسات الأمان:** يُمرّر Nginx ترويسات X-Forwarded-For و X-Real-IP لتمكين الـ API من تسجيل عنوان IP الحقيقي للعميل.
- **تثبيت الملف للقراءة فقط:** يُنَبِّت ملف nginx.conf بصلاحيات القراءة فقط (ro:) لمنع أي تعديل عليه من داخل الحاوية.

```
volumes:
- ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro
```

5.5 استخدام صورة أساسية خفيفة (Minimal Base Image)

اختيار صورة python:3.13-slim بدلاً من الصورة الكاملة يُقلّل مساحة الهجوم الأمنية (Attack Surface) عبر تقليل عدد الحزم والأدوات المثبتة مسبقاً في الصورة. كذلك استخدام postgres:17-alpine لقاعدة البيانات يتبع نفس المبدأ، حيث أن توزيع Alpine Linux تُعرّف بصغر حجمها وتقليلها للمكونات غير الضرورية.

5.6 تثبيت إصدارات الاعتماديات (Dependency Pinning)

تم تحديد إصدارات محددة لجميع مكتبات Python في ملف requirements.txt باستخدام عامل المطابقة

الدقيقة(==) ، مما يمنع تثبيت إصدارات أحدث غير مُختبرة تلقائياً ويضمن إعادة إنتاج (Reproducibility) البيئة بشكل مُطابق.

6. معمارية الشبكة (Network Architecture)

6.1 إعداد الشبكة في Docker Compose

يُعرّف ملف `compose.yaml` شبكة `bridge` مُخصّصة واحدة باسم `app-network` :

```
networks:
  app-network:
    driver: bridge
```

جميع الخدمات الثلاث (`nginx` ، `api` ، `database`) مُتصلة بهذه الشبكة:

```
services:
  nginx:
    networks:
      - app-network
  api:
    networks:
      - app-network
  database:
    networks:
      - app-network
```

6.2 آلية عمل الشبكة الداخلية

توفّر شبكة `bridge` المُخصّصة في Docker عدة ميزات مهمة:

اكتشاف الخدمات عبر DNS الداخلي: يُوفّر Docker خادم DNS مُدمجاً داخل الشبكات المُخصّصة (User-Defined Networks). يُمكن هذا كل حاوية من الوصول إلى الحاويات الأخرى باستخدام اسم

الخدمة (Service Name) بدلاً من عنوان IP. على سبيل المثال:

- يستخدم Nginx الاسم `api` للوصول إلى خدمة الـ

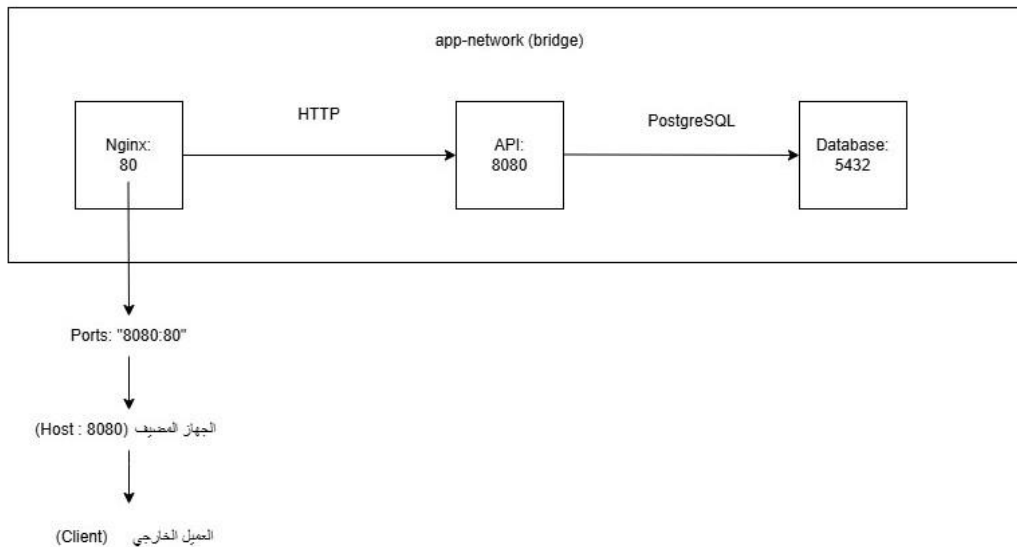
`upstream backend { server api:8000; } Backend: proxy_pass http://backend`

- يستخدم الـ API الاسم `database` للاتصال بـ `PostgreSQL: host=database port=5432`

العزل عن الشبكات الأخرى: الحاويات المُتصلة بشبكة `app-network` فقط يمكنها التواصل مع بعضها

البعض. لا يمكن لأي حاوية خارج هذه الشبكة الوصول إلى الخدمات الداخلية.

6.3 نموذج تدفق حركة البيانات (Traffic Flow Model)



6.4 عزل قاعدة البيانات عن الإنترنت

من أهم القرارات المعمارية في هذا المشروع هو عزل قاعدة البيانات تماماً عن الوصول الخارجي. تحقق ذلك من خلال:

1. عدم نشر المنفذ 5432: استخدام expose بدلاً من ports يجعل منفذ PostgreSQL متاحاً فقط داخل شبكة app-network.
2. عدم وجود اتصال مباشر: لا يمكن لأي عميل خارجي الاتصال بقاعدة البيانات مباشرة؛ فالوصول الوحيد يكون عبر طبقة الـ API التي تُطبّق منطق الأعمال والتحقق من صحة البيانات.
3. نفس المبدأ ينطبق على الـ API: لا يُنشر المنفذ 8080 على الجهاز المضيف، مما يُجبر جميع الطلبات على المرور عبر Nginx.

6.5 أدلة بصرية

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker network inspect taskmanagementrestservice_app-network --format "{{range .Containers}}{{.Name}}{{println}}{{end}}"
```

task-api
task-database
task-nginx

شرح الصورة 03: توضح هذه الصورة مُخرَج أمر فحص شبكة Docker الذي يعرض قائمة الحاويات المتصلة بشبكة app-network وعناوين IP الداخلية المُخصَّصة لكل منها، مما يُثبت أن جميع الخدمات الثلاث تعمل على نفس الشبكة المعزولة.

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
0fb3c1547960	nginx:latest	task-nginx	"/docker-entrypoint.s..."	10 minutes ago	Up 10 minutes	0.0.0.0:80
80->80/tcp, [::]:8080->80/tcp						
1fbbf835ea5f	task-management-api:v1.0.0	task-api	"uvicorn api.main:ap..."	10 minutes ago	Up 10 minutes (healthy)	8000/tcp
b6195e8b3fe6	postgres:17-alpine	task-database	"docker-entrypoint.s..."	10 minutes ago	Up 10 minutes (healthy)	5432/tcp

شرح الصورة 04: توضح هذه الصورة التحقق من عزل المنافذ الداخلية، حيث يظهر أن المنافذ 8080 فقط (الخاص بـ Nginx) هو المنشور على الجهاز المضيف، بينما منافذ الـ API (8000) وقاعدة البيانات (5432) مُقيّدة داخل شبكة Docker فقط.

7. معمارية التخزين (Storage Architecture)

7.1 أنواع التخزين المُستخدمة

يستخدم المشروع نوعين من آليات تخزين Docker:

7.1.1 (التخزين المُسمّى) Named Volumes

تم تعريف Named Volume باسم db-data لتخزين بيانات قاعدة PostgreSQL:

```
volumes:
  db-data:

services:
  database:
    volumes:
      - db-data:/var/lib/postgresql/data
```

خصائص Named Volumes:

- تُدار بالكامل بواسطة Docker Engine وتُخزَّن في مسار داخلي يُديره Docker (عادةً /var/lib/docker/volumes/ على Linux).
- تستمر بعد إيقاف أو حذف الحاوية (docker compose down).
- تُحذف فقط عند استخدام الأمر docker compose down -v أو docker volume rm.
- مثالية لتخزين بيانات قواعد البيانات التي يجب أن تستمر عبر دورات حياة الحاويات.
- يمكن فحصها باستخدام docker volume inspect.

7.1.1 (التثبيت المباشر) Bind Mounts

تم استخدام Bind Mounts لتنشيط ملفات التكوين من الجهاز المضيف إلى داخل الحاويات:

```
services:
  nginx:
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.conf:ro

  database:
    volumes:
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
```

خصائص Bind Mounts:

- تربط مساراً محدداً على الجهاز المضيف بمسار داخل الحاوية.
- تُستخدم هنا لملفات التكوين (Configuration Files) التي يحتاج المطور لتعديلها من خارج الحاوية.
- إضافة العلامة : ro (Read-Only) تمنع الحاوية من تعديل هذه الملفات، مما يُعزز الأمان.
- ملف `init.sql` يُنشأ في مجلد `/docker-entrypoint-initdb.d` الخاص بـ PostgreSQL، والذي يُنفذ تلقائياً عند تهيئة قاعدة البيانات لأول مرة فقط.

7.2 مقارنة بين نوعي التخزين المُستخدمين

المعيار	Named Volume (db-data)	Bind Mount (nginx.conf, init.sql)
الغرض	تخزين بيانات قاعدة البيانات	تنشيط ملفات التكوين
الاستمرارية	تستمر بعد حذف الحاوية	مرتبطة بملفات الجهاز المضيف
الإدارة	يديرها Docker	يديرها المستخدم
الصلاحيات	قراءة وكتابة	قراءة فقط (ro)
حالة الاستخدام	بيانات ديناميكية	ملفات ثابتة

8. التحقق من استمرارية البيانات (Persistence Verification)

8.1 أهمية التحقق

استمرارية البيانات (Data Persistence) هي مطلب جوهري لأي نظام يعتمد على قاعدة بيانات. في بيئة Docker، تُفقد جميع البيانات المخزنة داخل نظام ملفات الحاوية (Container Filesystem) عند حذف الحاوية، لأن نظام الملفات هذا مؤقت بطبيعته (Ephemeral). لذلك، يجب التحقق من أن Named Volume يحفظ البيانات بشكل مستقل عن دورة حياة الحاوية.

8.2 سيناريو التحقق

تم اتباع الخطوات التالية لإثبات استمرارية البيانات:

الخطوة 1 — إضافة بيانات: تم إرسال طلب POST لإنشاء مهمة جديدة في قاعدة البيانات عبر الـ API:

```
curl -X POST http://localhost:8080/api/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Persistence Test","description":"Testing data
persistence","status":"Pending","due_date":"2026-09-25"}'
```

ثم تم التحقق من وجود البيانات عبر طلب GET:

```
curl http://localhost:8080/api/tasks
```

الخطوة 2 — إيقاف وحذف الحاويات: تم إيقاف جميع الحاويات وحذفها باستخدام:

```
docker compose down
```

ملاحظة: الأمر `docker compose down` يحذف الحاويات والشبكات لكنه لا يحذف الـ `Named Volumes`.

الخطوة 3 — إعادة التشغيل والتحقق: تم إعادة تشغيل النظام بالكامل:

```
docker compose up -d
```

ثم تم الاستعلام عن البيانات مرة أخرى:

```
curl http://localhost:8080/api/tasks
```

النتيجة: البيانات التي تم إدخالها قبل إيقاف الحاويات ما زالت موجودة، مما يُثبت أن `Named Volume (db-data)` يحفظ بيانات PostgreSQL بشكل دائم ومستقل عن دورة حياة الحاويات.

8.3 فحص Volume بشكل مباشر

يمكن فحص تفاصيل الـ Volume باستخدام:

```
docker volume inspect taskmanagementrestservice_db-data
```

يعرض هذا الأمر معلومات مثل مسار التخزين الفعلي (Mountpoint) على الجهاز المضيف، وتاريخ الإنشاء، والمشغل (Driver) المستخدم.

8.4 التمييز المهم

`docker compose down` # يحذف الحاويات والشبكات فقط — البيانات تبقى
`docker compose down -v` # `Volumes` يحذف الحاويات والشبكات والـ — البيانات تُفقد

8.5 أدلة بصرية

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -X POST "http://localhost:8080/api/tasks" -H "Content-Type: application/json" --data-binary "@task.json"
{"title":"Compose Test Task","description":"Testing the final Docker Compose stack","status":"Pending","due_date":"2026-09-25","id":1}
```

شرح الصورة 05: توضح هذه الصورة إرسال طلب POST عبر Nginx لإنشاء مهمة جديدة في قاعدة البيانات كخطوة أولى في اختبار استمرارية البيانات.

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i "http://localhost:8080/api/tasks"
HTTP/1.1 200 OK
Server: nginx/1.31.3
Date: Wed, 16 Sep 2026 17:02:45 GMT
Content-Type: application/json
Content-Length: 136
Connection: keep-alive

[{"title":"Compose Test Task","description":"Testing the final Docker Compose stack","status":"Pending","due_date":"2026-09-25","id":1}]
```

شرح الصورة 06: توضح هذه الصورة نتيجة الاستعلام عن البيانات بعد إيقاف جميع الحاويات وحذفها ثم إعادة تشغيلها، حيث يظهر أن المهام المسجلة سابقاً ما زالت موجودة، مما يُثبت فعالية Named Volume في الحفاظ على استمرارية البيانات.

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker volume ls
DRIVER      VOLUME NAME
local       task-db-data
local       taskmanagementrestservice_db-data
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker volume inspect taskmanagementrestservice_db-data
[
  {
    "CreatedAt": "2026-09-16T16:53:27Z",
    "Driver": "local",
    "Labels": {
      "com.docker.compose.config-hash": "e41f72e6e882f932a15c0ee7cb0e9ba247ee69efed30049d415be93219da23ed",
      "com.docker.compose.project": "taskmanagementrestservice",
      "com.docker.compose.version": "5.3.1",
      "com.docker.compose.volume": "db-data"
    },
    "Mountpoint": "/var/lib/docker/volumes/taskmanagementrestservice_db-data/_data",
    "Name": "taskmanagementrestservice_db-data",
    "Options": null,
    "Scope": "local"
  }
]
```

شرح الصورة 07: توضح هذه الصورة مُخْرَج أمر `docker volume inspect` الذي يعرض تفاصيل (db-data) Named Volume بما في ذلك مسار التخزين الفعلي (Mountpoint) على الجهاز المُضيف ونوع المشغل (Driver) المُستخدم.

9. سيناريوهات الفشل والتعافي (Failure-Mode Scenarios)

9.1 أهمية اختبار الفشل

في الأنظمة الموزعة والمبنية على الحاويات، يُعدّ اختبار سيناريوهات الفشل جزءاً أساسياً من ضمان الجودة (Quality Assurance). يهدف هذا القسم إلى دراسة سلوك النظام عند توقف أحد مكوناته بشكل مفاجئ، وكيفية تعافي النظام بعد استعادة المكون المتوقف.

9.2 الحالة أ — توقف حاوية الـ API

سيناريو المحاكاة:

```
docker compose stop api
```

السلوك المُلاحظ:

- Nginx يظل يعمل ويستقبل الطلبات .
- عند إرسال طلب HTTP عبر Nginx، يحاول Nginx توجيهه إلى api:8000، لكن الخدمة غير متاحة.
- يُعيد Nginx رمز الخطأ 502 Bad Gateway للعميل، مما يُشير إلى أن الوكيل العكسي يعمل لكنه لا يستطيع الوصول إلى الخدمة الخلفية (Upstream) .
- قاعدة البيانات تظل تعمل بشكل طبيعي ولا تتأثر بتوقف الـ API.

استعادة الخدمة:

```
docker compose start api
```

بعد إعادة تشغيل حاوية الـ API، يعود النظام للعمل بشكل طبيعي دون الحاجة لإعادة تشغيل Nginx أو قاعدة البيانات. يتمكن Nginx من اكتشاف عودة الخدمة تلقائياً عبر آلية DNS الداخلية لـ Docker.

9.3 الحالة ب — توقف حاوية قاعدة البيانات

سيناريو المحاكاة:

```
docker compose stop database
```

السلوك المُلاحظ:

- Nginx والـ API يظلان يعملان من حيث الحالة التشغيلية.
- عند إرسال طلب يتطلب الوصول إلى قاعدة البيانات (مثل GET /api/tasks)، يحاول الـ API فتح اتصال مع PostgreSQL عبر مكتبة psycopg، لكنه يفشل بسبب عدم توفر الخدمة.

- يُعيد الـ API رمز خطأ 500 Internal Server Error مع رسالة تُشير إلى فشل الاتصال بقاعدة البيانات.
 - طلبات الـ Health Check الخاصة بالـ API قد تفشل أيضاً إذا كانت تعتمد على الاتصال بقاعدة البيانات.
- استعادة الخدمة:

```
docker compose start database
```

بعد إعادة تشغيل حاوية قاعدة البيانات، يتمكن الـ API من إعادة إنشاء الاتصال تلقائياً عند الطلب التالي (لأن كل طلب يُنشئ اتصالاً جديداً عبر `get_connection()` . البيانات المُخزَّنة في `Named Volume` تبقى سليمة ومُتاحة.

9.4 الحالة ج — إعادة إنشاء حاوية الـ API بالقوة

سيناريو المحاكاة:

```
docker compose up -d --force-recreate api
```

السلوك المُلاحظ: يتم حذف حاوية الـ API الحالية وإنشاء حاوية جديدة بالكامل. بعد إعادة الإنشاء، يعود النظام للعمل بشكل طبيعي، وتبقى جميع بيانات المهام موجودة لأنها مُخزَّنة في قاعدة البيانات (على `Named Volume`) وليس داخل حاوية الـ API نفسها. هذا يُثبت أن الـ API هو خدمة بلا حالة (Stateless Service) .

9.5 آليات التعافي المُعرَّفة

تم تعريف آليتين رئيسيتين في `compose.yaml` لضمان التسلسل الصحيح عند بدء التشغيل:

depends_on & condition: service_healthy:

```
api:
  depends_on:
    database:
      condition: service_healthy

nginx:
  depends_on:
    api:
      condition: service_healthy
```

هذا يضمن أن:

1. لا تبدأ خدمة الـ API إلا بعد أن تُصبح قاعدة البيانات جاهزة (healthy) — وليس مجرد أن الحاوية بدأت.

2. لا يبدأ Nginx إلا بعد أن يُصبح الـ API جاهزاً (healthy).

لاكتشاف الجاهزية healthcheck :

```
database:
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s

api:
  healthcheck:
    test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('http://localhost:8000/')"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 10s
```

9.6 أدلة بصرية

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose stop api
[+] stop 1/1
✔ Container task-api Stopped 0.4s
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i "http://localhost:8080/api/tasks"
HTTP/1.1 502 Bad Gateway
Server: nginx/1.31.3
Date: Wed, 16 Sep 2026 17:12:02 GMT
Content-Type: text/html
Content-Length: 157
Connection: keep-alive

<html>
<head><title>502 Bad Gateway</title></head>
<body>
<center><h1>502 Bad Gateway</h1></center>
<hr><center>nginx/1.31.3</center>
</body>
</html>
```

شرح الصورة 08: توضح هذه الصورة سيناريو توقف حاوية الـ API، حيث يظهر استجابة Nginx بخطأ 502 Bad Gateway عند محاولة الوصول إلى الـ API بعد إيقافه، مما يُثبت أن Nginx يتعامل مع فقدان الخدمة الخلفية بشكل أنيق (Graceful Degradation).

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose stop database
[+] stop 1/1
✓Container task-database Stopped 0.2s
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i "http://localhost:8080/api/tasks"
HTTP/1.1 500 Internal Server Error
Server: nginx/1.31.3
Date: Wed, 16 Sep 2026 17:14:20 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 21
Connection: keep-alive
Internal Server Error
```

شرح الصورة 09: توضح هذه الصورة سيناريو توقف حاوية قاعدة البيانات، حيث يظهر فشل طلبات الـ API التي تتطلب الوصول إلى PostgreSQL بسبب عدم توفر اتصال قاعدة البيانات.

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose up -d --force-recreate api
[+] up 2/2
✓Container task-database Healthy 1.0s
✓Container task-api Started 1.1s
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose ps
NAME                PORTS                IMAGE                COMMAND                SERVICE    CREATED        STATUS
task-api            8000/tcp             task-management-api:v1.0.0    "uvicorn api.main:ap..."    api        11 seconds ago Up 9 seconds (healthy)
task-database       5432/tcp             postgres:17-alpine          "docker-entrypoint.s..."    database   11 minutes ago Up About a minute (health)
task-nginx          0.0.0.0:8080->80/tcp, [::]:8080->80/tcp    nginx:latest            "/docker-entrypoint..."    nginx      11 minutes ago Up 10 minutes
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i "http://localhost:8080/api/tasks"
HTTP/1.1 200 OK
Server: nginx/1.31.3
Date: Wed, 16 Sep 2026 17:16:49 GMT
Content-Type: application/json
Content-Length: 136
Connection: keep-alive
[{"title": "Compose Test Task", "description": "Testing the final Docker Compose stack", "status": "Pending", "due_date": "2026-09-25", "id": 1}]
```

شرح الصورة 10: توضح هذه الصورة نتيجة إعادة إنشاء حاوية الـ API بالقوة (--force-recreate)، حيث يظهر أن البيانات ما زالت موجودة بعد إعادة الإنشاء، مما يُثبت أن الـ API خدمة بلا حالة وأن البيانات تعتمد على التخزين الدائم في Named Volume.

10. سجل الحاويات وإدارة النسخ (Registry & Versioning)

10.1 استراتيجية تسمية الصور (Image Tagging Strategy)

تم اعتماد استراتيجية تسمية مزدوجة للصور:

الغرض	الوسم (Tag)
وسم ثابت (Immutable Tag) يُشير إلى إصدار مُحدّد وقابل لإعادة الإنتاج	v1.0.0

الغرض	الوسم (Tag)
وسم مُتحرِّك (Mutable Tag) يُشير دائماً إلى أحدث إصدار	latest

بناء الصورة محلياً:

```
docker build -f api/Dockerfile -t task-management-api:v1.0.0 .
```

إعادة التسمية للرفع إلى **Docker Hub**:

```
docker tag task-management-api:v1.0.0 osama2sv/task-management-api:v1.0.0
docker tag task-management-api:latest osama2sv/task-management-api:latest
```

10.2 الرفع إلى Docker Hub

تم رفع الصورة بكلا الوسمين إلى مستودع Docker Hub:

```
docker push osama2sv/task-management-api:v1.0.0
docker push osama2sv/task-management-api:latest
```

مستودع **Docker Hub**:

```
docker.io/osama2sv/task-management-api
```

10.3 السحب والتحقق

تم التحقق من إمكانية سحب الصورة من Docker Hub وتشغيلها:

```
docker pull osama2sv/task-management-api:v1.0.0
docker pull osama2sv/task-management-api:latest
```

10.4 أهمية نظام الإصدارات

- **الوسم v1.0.0**: يسمح بالرجوع إلى إصدار مُحدَّد في أي وقت، وهو ضروري في بيئات الإنتاج حيث تُطلب القدرة على التراجع (Rollback).
- **الوسم latest**: يُسهِّل على المطورين سحب أحدث إصدار دون الحاجة لمعرفة رقم الإصدار الدقيق. ومع ذلك، لا يُنصح باستخدامه في بيئات الإنتاج بسبب عدم ثباته.

10.5 أدلة بصرية

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker push osama2sv/task-management-api:v1.0.0
The push refers to repository [docker.io/osama2sv/task-management-api]
219950e2bea4: Layer already exists
2c868586e995: Layer already exists
9b83eae799b8: Layer already exists
425d1038d5f6: Layer already exists
5bbcb35ea63e: Layer already exists
4329a4aff707: Layer already exists
370c1ae97976: Already exists
6310eb16bf42: Layer already exists
c4bdfa8c14a6: Layer already exists
237e12970bdc: Layer already exists
v1.0.0: digest: sha256:9cd378396ea5e287c9c5966a17f396c3f191a137d7d7450f28b6c3f62d3b6af6 size: 856
```

شرح الصورة 11: توضح هذه الصورة عملية رفع (Push) صورة الـ API بالوسم v1.0.0 إلى مستودع Docker Hub، حيث تظهر طبقات الصورة التي يتم رفعها وحالة اكتمال العملية.

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker image ls task-management-api
INFO → U In Use
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
task-management-api:latest	9cd378396ea5	254MB	60.7MB	U
task-management-api:v1.0.0	9cd378396ea5	254MB	60.7MB	U

شرح الصورة 12: توضح هذه الصورة قائمة الصور المحلية (docker image ls) مع الوسوم (Tags) المختلفة المُعرّفة لصورة task-management-api، بما في ذلك v1.0.0 وlatest والمستودع البعيد osama2sv/task-management-api.

11. أدلة الاختبار (Testing Evidence)

يُقدِّم هذا القسم أدلة شاملة وموثقة بصرياً تُغطي جميع جوانب اختبار النظام، بما في ذلك اختبار واجهات (API Endpoints)، وحالات الخطأ (Error Handling)، وعمليات Docker المختلفة، وسجلات الحاويات.

11.1 اختبار واجهات API — العمليات الأساسية (CRUD Operations)

11.1.1 استرجاع جميع المهام — GET /api/tasks

الطلب:

```
curl http://localhost:8080/api/tasks
```

الاستجابة المتوقعة: رمز الحالة 200 OK مع مصفوفة JSON تحتوي على جميع المهام المُسجَّلة.

Server response

Code	Details
200	Response body <pre>[{ "title": "Prepare Project Report", "description": "Complete the technical report", "status": "Pending", "due_date": "2026-09-21", "id": 1 }, { "title": "Prepare Project Report", "description": "Complete the technical report", "status": "Pending", "due_date": "2026-09-21", "id": 3 }]</pre> Download

شرح الصورة 13: توضح هذه الصورة الاستجابة الناجحة لـ Endpoint الخاص باسترجاع جميع المهام (GET /api/tasks) مع رمز الحالة 200 OK، حيث يُعيد الخادم مصفوفة JSON تحتوي على قائمة المهام المُخزَّنة في قاعدة البيانات.

11.1.2 إنشاء مهمة جديدة — POST /api/tasks

الطلب:

```
curl -X POST http://localhost:8080/api/tasks \
-H "Content-Type: application/json" \
-d '{"title":"Example Task","description":"Task
description","status":"Pending","due_date":"2026-09-25"}'
```

الاستجابة المتوقعة: رمز الحالة 201 Created مع بيانات المهمة المنشأة بما في ذلك الـ id المُخصَّص تلقائياً.

Server response	
Code	Details
201	Response body <pre>{ "title": "Evidence Task", "description": "Task created for project testing", "status": "Pending", "due_date": "2026-09-21", "id": 4 }</pre> Download

شرح الصورة 14: توضح هذه الصورة الاستجابة الناجحة لـ Endpoint الخاص بإنشاء مهمة جديدة (POST /api/tasks) مع رمز الحالة 201 Created، حيث يُعيد الخادم بيانات المهمة المنشأة بنجاح مع الـ id المُعَيَّن تلقائياً.

11.1.3 استرجاع مهمة بالمعرف — GET /api/tasks/{id}

الطلب:

```
curl http://localhost:8080/api/tasks/1
```

الاستجابة المتوقعة: رمز الحالة 200 OK مع بيانات المهمة المُحدَّدة.

Server response	
Code	Details
200	Response body <pre>{ "title": "Evidence Task", "description": "Task created for project testing", "status": "Pending", "due_date": "2026-09-21", "id": 4 }</pre> Download

شرح الصورة 15: توضح هذه الصورة الاستجابة الناجحة لـ Endpoint الخاص باسترجاع مهمة واحدة بالمعرف (GET /api/tasks/{id}) مع رمز الحالة 200 OK، حيث يُعيد الخادم بيانات المهمة المطلوبة بتنسيق JSON.

11.1.4 تحديث مهمة — PUT /api/tasks/{id}

الطلب:

```
curl -X PUT http://localhost:8080/api/tasks/1 \
-H "Content-Type: application/json" \
-d '{"title": "Updated Task", "description": "Updated description", "status": "In Progress", "due_date": "2026-09-25"}'
```

الاستجابة المتوقعة: رمز الحالة 200 OK مع بيانات المهمة المُحدّثة.

Server response

Code	Details
200	Response body <pre>{ "title": "Updated Task", "description": "Task updated for project testing", "status": "In Progress", "due_date": "2026-09-21", "id": 1 }</pre> Download

شرح الصورة 16: توضح هذه الصورة الاستجابة الناجحة لـ Endpoint الخاص بتحديث مهمة موجودة (PUT /api/tasks/{id}) مع رمز الحالة 200 OK، حيث يظهر أن بيانات المهمة تم تحديثها بنجاح (مثل تغيير الحالة من Pending إلى In Progress).

11.1.5 حذف مهمة — DELETE /api/tasks/{id}

الطلب:

```
curl -X DELETE http://localhost:8080/api/tasks/1
```

الاستجابة المتوقعة: رمز الحالة 204 No Content بدون جسم استجابة (Empty Body).

Server response

Code	Details
204	Response headers <pre>content-type: application/json date: Tue, 15 Sep 2026 18:57:12 GMT server: uvicorn</pre>

شرح الصورة 17: توضح هذه الصورة الاستجابة الناجحة لـ Endpoint الخاص بحذف مهمة (DELETE /api/tasks/{id}) مع رمز الحالة 204 No Content، مما يؤكد حذف المهمة بنجاح.

11.2 اختبار حالات الخطأ (Error Handling)

11.2.1 الوصول إلى مهمة غير موجودة — 404 Not Found

الطلب:

```
curl http://localhost:8080/api/tasks/999
```

الاستجابة المتوقعة: رمز الحالة 404 Not Found مع رسالة "Task not found".

Server response	
Code	Details
404 Undocumented	Error: Not Found Response body <pre>{ "detail": "Task not found" }</pre>  Download

شرح الصورة 18: توضح هذه الصورة استجابة النظام عند محاولة الوصول إلى مهمة غير موجودة (GET /api/tasks/999)، حيث يُعيد الخادم رمز الحالة 404 Not Found مع رسالة خطأ واضحة تُفيد بأن المهمة غير موجودة.

11.2.2 إرسال حالة غير صالحة — 422 Unprocessable Entity

الطلب:

```
curl -X POST http://localhost:8080/api/tasks \  
-H "Content-Type: application/json" \  
-d '{"title":"Test","status":"InvalidStatus"}'
```

الاستجابة المتوقعة: رمز الحالة 422 Unprocessable Entity مع تفاصيل خطأ التحقق من صحة البيانات (Validation Error).

Server response	
Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": { { "type": "enum", "loc": ["body", "status"], "msg": "Input should be 'Pending', 'In Progress' or 'Completed'", "input": "Finished", "ctx": { "expected": "'Pending', 'In Progress' or 'Completed'" } } } }</pre>  Download

شرح الصورة 19: توضح هذه الصورة استجابة النظام عند إرسال قيمة حالة غير صالحة (InvalidStatus) لا تتطابق مع القيم المُعرَّفة في TaskStatus Enum (Pending Progress, Completed)، حيث يُعيد FastAPI رمز الحالة 422 Unprocessable Entity مع تفاصيل خطأ التحقق (Validation Error Details).

11.3 اختبار عمليات بناء Docker وفحص الصور

11.3.1 بناء الصورة

```
(.env) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker build -f .\api\Dockefile -t task-management-api:v1.0.0 .
[+] Building 115.4s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 387B
=> [internal] load metadata for docker.io/library/python:3.13-slim
=> [internal] load .dockerignore
=> => transferring context: 142B
=> [1/6] FROM docker.io/library/python:3.13-slim@sha256:9d2e5553305c7c7b0097999bb17187c69b921ccd6bc9d40e4bb5ebe 79.0s
=> => resolve docker.io/library/python:3.13-slim@sha256:9d2e5553305c7c7b0097999bb17187c69b921ccd6bc9d40e4bb5ebe6 0.0s
=> sha256:9b83eae799b831086df12d4da65c2edb0e6c713caa4312455bc8ee4b1a256c39 249B / 249B 0.3s
=> sha256:237e12970bdcabcedd95dab0f8dc59cda22b450156ee0bd01c58e0cf3b9c43885 11.91MB / 11.91MB 41.0s
=> sha256:5bbcb35ea63ee8dd9b96c06b0f78b33c9a32e06e5b14e1bfeb46b498a93edb5d 4.27MB / 4.27MB 39.7s
=> sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 29.79MB / 29.79MB 78.0s
=> extracting sha256:6310eb16bf4251731feab01e8f633bf5e2d75a657ccad97f420b1f83cce457be 0.6s
=> extracting sha256:5bbcb35ea63ee8dd9b96c06b0f78b33c9a32e06e5b14e1bfeb46b498a93edb5d 0.1s
=> extracting sha256:237e12970bdcabcedd95dab0f8dc59cda22b450156ee0bd01c58e0cf3b9c43885 0.3s
=> extracting sha256:9b83eae799b831086df12d4da65c2edb0e6c713caa4312455bc8ee4b1a256c39 0.0s
=> [internal] load build context
=> => transferring context: 14.04kB
=> [2/6] WORKDIR /app
=> [3/6] COPY api/requirements.txt ./requirements.txt
=> [4/6] RUN pip install --no-cache-dir -r requirements.txt 31.4s
=> [5/6] COPY api/ ./api/
=> [6/6] RUN groupadd --system appgroup && useradd --system --gid appgroup appuser
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:84d64f5b0d6bc64722c6cebd05e8bb0e02ff9f167eac326ce556cb2098019669 0.0s
=> => exporting config sha256:3d2af83ca45e02bd5acced52051bd258755f2e7772e33818fc3de01eced3c159 0.0s
=> => exporting attestation manifest sha256:ce40f132a4a88bf7de3d04f978de9435b0b9b349ef35759d5d3d850bb9a85459 0.0s
=> => exporting manifest list sha256:54cf622dc4aa5179e87dd8d2c7b451c895beb7de31e333d77247ca330d2ffde7 0.0s
=> => naming to docker.io/library/task-management-api:v1.0.0
=> => unpacking to docker.io/library/task-management-api:v1.0.0 0.4s

View build details: docker-desktop://dashboard/build/desktop-linux/desktop-linux/at8y7er3y7wc6e5gs8mjebf5n
(.env) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service>
```

شرح الصورة 20: توضح هذه الصورة مُخرَج أمر docker build أثناء بناء صورة الـ API بالوسم v1.0.0، حيث تظهر مراحل البناء المتسلسلة لكل طبقة (Layer) في ملف Dockerfile.

11.3.2 قائمة الصور المحلية

Image Name	Image ID	Size	Size on Disk
task-management-api:v1.0.0	54cf622dc4aa	254MB	60.7MB

شرح الصورة 21: توضح هذه الصورة مُخرَج أمر docker image ls الذي يعرض قائمة الصور المحلية بما في ذلك صورة task-management-api وحجمها.

11.3.3 تاريخ طبقات الصورة

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker image history task-management-api:v1.0.0
```

IMAGE	CREATED	CREATED BY	SIZE	COMMENT
54cf622dc4aa	8 minutes ago	CMD ["uvicorn" "api.main:app" "--host" "0.0.0.0"]	0B	buildkit.dockerfile.v0
<missing>	8 minutes ago	EXPOSE [8000/tcp]	0B	buildkit.dockerfile.v0
<missing>	8 minutes ago	USER appuser	0B	buildkit.dockerfile.v0
<missing>	8 minutes ago	RUN /bin/sh -c groupadd --system appgroup	41kB	buildkit.dockerfile.v0
<missing>	8 minutes ago	COPY api/ ./api/ # buildkit	57.3kB	buildkit.dockerfile.v0
<missing>	8 minutes ago	RUN /bin/sh -c pip install --no-cache-dir -r requirements.txt	52.5MB	buildkit.dockerfile.v0
<missing>	9 minutes ago	COPY api/requirements.txt ./requirements.txt	12.3kB	buildkit.dockerfile.v0
<missing>	9 minutes ago	WORKDIR /app	8.19kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	CMD ["python3"]	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; for src in idle3 p...	16.4kB	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; savedAptMark="\$(a...	40.4MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PYTHON_SHA256=1e66a7945a48390ee4c2a4268a...	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PYTHON_VERSION=3.13.15	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV GPG_KEY=7169605F62C751356D054A26A821E680...	0B	buildkit.dockerfile.v0
<missing>	2 weeks ago	RUN /bin/sh -c set -eux; apt-get update; a...	13.2MB	buildkit.dockerfile.v0
<missing>	2 weeks ago	ENV PATH=/usr/local/bin:/usr/local/sbin:/usr...	0B	buildkit.dockerfile.v0
<missing>	3 weeks ago	# debian.sh --arch 'amd64' out/ 'trixie' '@1...	87.5MB	debuerreotype 0.17

شرح الصورة 22: توضح هذه الصورة مُخَرَج أمر docker image history الذي يعرض تفاصيل كل طبقة في صورة task-management-api:v1.0.0، بما في ذلك حجم كل طبقة والأمر الذي أنشأها، مما يُساعد في تحليل وتحسين حجم الصورة.

11.3.4 فحص تفاصيل الصورة

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker image inspect task-management-api:v1.0.0 --format "ID={{.Id}} | Created={{.Created}} | Size={{.Size}} | Architecture={{.Architecture}} | OS={{.Os}}"
```

ID=sha256:19998f9684440010b54f09f5122b4bc9b04ae2c5d912b28aab7c57ef9b8afe69 | Created=2026-09-16T21:43:39.342159345Z | Size=60704120 | Architecture=amd64 | OS=linux

شرح الصورة 23: توضح هذه الصورة مُخَرَج أمر docker image inspect الذي يعرض البيانات الوصفية (Metadata) للصورة بما في ذلك البيئة والمستخدم النشط والمنافذ المُعرَّضة.

11.4 اختبار تشغيل الحاويات

11.4.1 حالة تشغيل الحاوية

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
23892c1140b6	task-management-api:v1.0.0	"uvicorn api.main:ap..."	12 seconds ago	Up 11 seconds	0.0.0.0:8000->8000
/tcp, [::]:8000->8000/tcp	task-api-test				
85311a3dfd10	postgres:17-alpine	"docker-entrypoint.s..."	23 hours ago	Up 27 minutes	0.0.0.0:5432->5432
/tcp, [::]:5432->5432/tcp	task-postgres				

شرح الصورة 24: توضح هذه الصورة حالة تشغيل حاويات النظام، حيث يظهر أن جميع الحاويات الثلاث (task-nginx، task-api، task-database) في حالة تشغيل (Running) وجاهزة لاستقبال الطلبات.

11.4.2 التحقق من تشغيل المستخدم غير الجذري

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker exec task-api-test id
```

uid=999(appuser) gid=999(appgroup) groups=999(appgroup)

شرح الصورة 25: توضح هذه الصورة التحقق من أن تطبيق FastAPI يعمل داخل الحاوية بصلاحيات المستخدم appuser (وليس root)، مما يُثبت تطبيق ممارسة الأمان الخاصة بالتنشغيل كمستخدم غير جذري (Non-root User).

11.4.3 اتصال الـ API بقاعدة البيانات

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker exec task-api-test python -c "import urllib.request; print(urllib.request.urlopen('http://localhost:8000/api/tasks').read().decode())"
[{"title": "Updated Task", "description": "Task updated for project testing", "status": "In Progress", "due_date": "2026-09-21", "id": 1}, {"title": "Prepare Project Report", "description": "Complete the technical report", "status": "Pending", "due_date": "2026-09-21", "id": 3}]
```

شرح الصورة 26: توضح هذه الصورة نجاح اتصال حاوية الـ API بقاعدة البيانات PostgreSQL عبر الشبكة الداخلية app-network، حيث يتمكن التطبيق من تنفيذ عمليات CRUD بنجاح.

11.4.4 اختبار الـ API مباشرة من داخل Docker

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl http://localhost:8000/

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is
parsed.
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): y

StatusCode      : 200
StatusDescription : OK
Content          : {"message":"Task Management REST Service is running"}
RawContent       : HTTP/1.1 200 OK
                  Content-Length: 53
                  Content-Type: application/json
                  Date: Wed, 16 Sep 2026 16:19:20 GMT
                  Server: uvicorn

                  {"message":"Task Management REST Service is running"}
Forms           : {}
Headers         : {[Content-Length, 53], [Content-Type, application/json], [Date, Wed, 16 Sep 2026 16:19:20 GMT],
                  [Server, uvicorn]}
Images          : {}
InputFields     : {}
Links           : {}
ParsedHtml      : mshtml.HTMLDocumentClass
RawContentLength : 53
```

شرح الصورة 27: توضح هذه الصورة اختبار واجهات الـ API مباشرة من بيئة Docker، مع عرض الاستجابات الناجمة لعمليات CRUD الأساسية.

11.5 اختبار Nginx Reverse Proxy

11.5.1 توجيه الطلبات عبر Nginx

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker ps
CONTAINER ID   IMAGE          NAMES                COMMAND                  CREATED        STATUS        PORTS
97b345024a6d   nginx:latest   task-nginx-test      "/docker-entrypoint.s... 18 seconds ago Up 18 seconds 0.0.0.0:8080->80/t
cp, [::]:8080->80/tcp
0b29962ae039   task-managem... task-management-api:v1.0.0 "uvicorn api.main:ap..." 9 minutes ago  Up 9 minutes  0.0.0.0:8000->8000
/tcp, [::]:8000->8000/tcp
85311a3dfd10   postgres:17-alpine "docker-entrypoint.s..." 23 hours ago   Up 46 minutes  0.0.0.0:5432->5432
/tcp, [::]:5432->5432/tcp
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe http://localhost:8080/api/tasks
[{"title":"Updated Task","description":"Task updated for project testing","status":"In Progress","due_date":"2026-09-21","id":1}, {"title":"Prepare Project Report","description":"Complete the technical report","status":"Pending","due_date":"2026-09-21","id":3}]
```

شرح الصورة 28: توضح هذه الصورة اختبار توجيه الطلبات عبر Nginx Reverse Proxy على المنفذ 8080، حيث يظهر أن الطلبات يتم توجيهها بنجاح إلى خدمة الـ API الداخلية.

11.5.2 التحقق من صحة تكوين Nginx

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker exec task-nginx-test nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

الصورة 29: توضح هذه الصورة التحقق من صحة ملف تكوين Nginx (nginx.conf) باستخدام أمر اختبار التكوين، مما يؤكد أن إعدادات الوكيل العكسي والـ upstream معرفّة بشكل صحيح.

11.5.3 اختبار POST عبر Nginx

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i http://localhost:8080/api/tasks
HTTP/1.1 200 OK
Server: nginx/1.31.3
Date: Wed, 16 Sep 2026 16:57:17 GMT
Content-Type: application/json
Content-Length: 2
Connection: keep-alive
[]
```

شرح الصورة 30: توضح هذه الصورة اختبار إرسال طلبات API عبر Nginx في بيئة Docker Compose، حيث تُظهر أن الوكيل العكسي يُوجّه الطلبات بشكل صحيح إلى خدمة الـ Backend.

11.5.4 التحقق من البيانات بعد POST عبر Nginx

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i "http://localhost:8080/api/tasks"
HTTP/1.1 200 OK
Server: nginx/1.31.3
Date: Wed, 16 Sep 2026 17:02:45 GMT
Content-Type: application/json
Content-Length: 136
Connection: keep-alive
[{"title":"Compose Test Task","description":"Testing the final Docker Compose stack","status":"Pending","due_date":"2026-09-25","id":1}]
```

شرح الصورة 31: توضح هذه الصورة استرجاع البيانات عبر Nginx بعد إنشائها بطلب POST، مما

يُثبت أن سلسلة الطلبات الكاملة

(Client → Nginx → API → Database → API → Nginx → Client)

متكامل.

11.6 اختبار عزل المنافذ والأمان

11.6.1 التحقق من المنافذ المنشورة

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
0fb3c1547960	nginx:latest		"/docker-entrypoint..."	10 minutes ago	Up 10 minutes	0.0.0.0:80
80->80/tcp, [::]:8080->80/tcp	task-nginx					
1fbff835ea5f	task-management-api:v1.0.0	task-api	"uvicorn api.main:ap..."	10 minutes ago	Up 10 minutes (healthy)	8000/tcp
b6195e8b3fe6	postgres:17-alpine	task-database	"docker-entrypoint.s..."	10 minutes ago	Up 10 minutes (healthy)	5432/tcp

شرح الصورة 32: توضح هذه الصورة التحقق النهائي من المنافذ المنشورة على الجهاز المضيف، حيث

يظهر أن المنفذ 8080 فقط (الخاص بـ Nginx) مُتاح خارجياً، بينما المنافذ الداخلية

(8000 للـ API و 5432 لقاعدة البيانات) معزولة داخل شبكة Docker.

11.6.2 تفاصيل تخطيط المنافذ

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker port task-api
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker port task-nginx
80/tcp -> 0.0.0.0:8080
80/tcp -> [::]:8080
```

شرح الصورة 33: توضح هذه الصورة تفاصيل تخطيط المنافذ (Port Mapping) لكل حاوية، مما

يؤكد أن استراتيجية العزل مُطبَّقة بشكل صحيح وأن الوصول الخارجي محصور في Nginx فقط.

11.7 اختبارات إضافية عبر Docker Hub

11.7.1 رفع الصورة بوسم latest

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker push osama2sv/task-management-api:latest
The push refers to repository [docker.io/osama2sv/task-management-api]
2c868586e995: Pushed
c4bdfa8c14a6: Pushed
6310eb16bf42: Pushed
370c1ae97976: Pushed
9b83eae799b8: Pushed
425d1038d5f6: Pushed
5bbcb35ea63e: Pushed
237e12970bdc: Pushed
219950e2bea4: Pushed
4329a4aff707: Pushed
latest: digest: sha256:9cd378396ea5e287c9c5966a17f396c3f191a137d7d7450f28b6c3f62d3b6af6 size: 856
```

شرح الصورة 34: توضح هذه الصورة عملية رفع (Push) صورة الـ API بالوسم latest إلى

مستودع Docker Hub، مع عرض حالة رفع الطبقات.

11.7.2 سحب الصورة بوسم v1.0.0

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker pull osama2sv/task-management-api:v1.0.0
v1.0.0: Pulling from osama2sv/task-management-api
Digest: sha256:9cd378396ea5e287c9c5966a17f396c3f191a137d7d7450f28b6c3f62d3b6af6
Status: Image is up to date for osama2sv/task-management-api:v1.0.0
docker.io/osama2sv/task-management-api:v1.0.0

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview osama2sv/task-management-api:v1.0.0
```

شرح الصورة 35: توضح هذه الصورة عملية سحب (Pull) صورة الـ API بالوسم v1.0.0 من مستودع Docker Hub، مما يُثبت أن الصورة مُتاحة للتنزيل والتشغيل من أي مكان.

11.7.3 سحب الصورة بوسم latest

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker pull osama2sv/task-management-api:latest
latest: Pulling from osama2sv/task-management-api
Digest: sha256:9cd378396ea5e287c9c5966a17f396c3f191a137d7d7450f28b6c3f62d3b6af6
Status: Image is up to date for osama2sv/task-management-api:latest
docker.io/osama2sv/task-management-api:latest

What's next:
View a summary of image vulnerabilities and recommendations → docker scout quickview osama2sv/task-management-api:latest
```

شرح الصورة 36: توضح هذه الصورة عملية سحب (Pull) صورة الـ API بالوسم latest من مستودع Docker Hub.

11.7.4 تشغيل الصورة المسحوبة من Docker Hub

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker run -d --name registry-api-test -p 8001:8000 osama2sv/task-management-api:v1.0.0
00 034c74e92f7b003bca39ebe66ad793278a6d8e00b411d51d86fb6d410d34c591
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker ps
CONTAINER ID   IMAGE                                NAMES      COMMAND                  CREATED        STATUS        PORTS
034c74e92f7b   osama2sv/task-management-api:v1.0.0  registry-api-test  "uvicorn api.main:ap..."  7 seconds ago  Up 6 seconds  0.0.0
034c74e92f7b   task-management-api:v1.0.0          task-api        "/docker-entrypoint.s..."  4 hours ago   Up 4 hours   8000/tcp
fe8f71ccc5a8   nginx:latest                        task-nginx      "/docker-entrypoint.s..."  4 hours ago   Up 4 hours    0.0.0
e5db44a7331f   postgres:17-alpine                 task-database    "docker-entrypoint.s..."  4 hours ago   Up 4 hours   5432/tcp
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i http://localhost:8001/
HTTP/1.1 200 OK
date: Wed, 16 Sep 2026 21:27:38 GMT
server: uvicorn
content-length: 53
content-type: application/json

{"message": "Task Management REST Service is running"}
```

شرح الصورة 37: توضح هذه الصورة اختبار تشغيل الصورة المسحوبة من Docker Hub والتحقق من أنها تعمل بشكل صحيح ومُطابق للصورة المحلية.

11.8 سجلات الحاويات (Container Logs)

```
(.venv) PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> docker compose logs --tail=20 nginx
task-nginx | /docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
task-nginx | /docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
task-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
task-nginx | 10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
task-nginx | 10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
task-nginx | /docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
task-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
task-nginx | /docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
task-nginx | /docker-entrypoint.sh: Configuration complete; ready for start up
task-nginx | 172.19.0.1 - - [16/Sep/2026:21:44:18 +0000] "GET /api/tasks HTTP/1.1" 200 136 "-" "curl/8.19.0"
```

شرح الصورة 38: توضح هذه الصورة سجلات حاوية (docker logs task-nginx) التي تعرض سجلات الوصول (Access Logs) للطلبات الواردة، بما في ذلك عناوين IP للعملاء وأكواد الحالة (Status Codes) والمسارات المطلوبة (Requested Paths).

11.9 اختبارات CRUD الإضافية

11.9.1 تحديث مهمة — اختبار إضافي

```
PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -X PUT "http://localhost:8080/api/tasks/2" -H
"Content-Type: application/json" --data-binary "@update-task.json"
{"title":"Updated PUT Test Task","description":"Task updated successfully using PUT","status":"In Progress","due_date":
2026-10-05","id":2}
```

شرح الصورة 39: توضح هذه الصورة اختبار إضافي لعملية تحديث مهمة (PUT /api/tasks/{id}) مع تغيير حقول متعددة، مما يؤكد أن عملية التحديث تعمل بشكل صحيح لجميع حقول المهمة.

11.9.2 حذف مهمة — اختبار إضافي

```
PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i -X DELETE "http://localhost:8080/api/tasks/
2"
HTTP/1.1 204 No Content
Server: nginx/1.31.3
Date: Thu, 17 Sep 2026 02:53:52 GMT
Content-Type: application/json
Connection: keep-alive
```

شرح الصورة 40: توضح هذه الصورة اختبار إضافي لعملية حذف مهمة (DELETE /api/tasks/{id}) ، مع التحقق من أن المهمة تم حذفها فعلاً من قاعدة البيانات.

11.10 اختبارات التحقق من صحة البيانات (Validation Testing)

11.10.1 طلب غير صالح — تحقق الإدخال

```
PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> @'
>> {
>>   "title": "",
>>   "description": "Invalid request test",
>>   "status": "Pending",
>>   "due_date": "2026-10-10"
>> }
>> '@ | Set-Content .\invalid-task.json
PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i -X POST "http://localhost:8080/api/tasks" -
H "Content-Type: application/json" --data-binary "@invalid-task.json"
HTTP/1.1 422 Unprocessable Content
Server: nginx/1.31.3
Date: Thu, 17 Sep 2026 02:55:00 GMT
Content-Type: application/json
Content-Length: 145
Connection: keep-alive
```

شرح الصورة 41: توضح هذه الصورة استجابة النظام عند إرسال طلب ببيانات غير صالحة أو ناقصة، حيث يقوم FastAPI بالتحقق التلقائي من صحة البيانات (Automatic Validation) عبر نماذج Pydantic ويُعيد رسائل خطأ مُفصَّلة تُوضِّح الحقول المفقودة أو غير الصالحة.

11.10.2 مورد غير موجود — 404

```
PS C:\Users\Osama\OneDrive\Desktop\Task Management REST Service> curl.exe -i "http://localhost:8080/api/tasks/9999"
HTTP/1.1 404 Not Found
Server: nginx/1.31.3
Date: Thu, 17 Sep 2026 02:56:19 GMT
Content-Type: application/json
Content-Length: 27
Connection: keep-alive

{"detail":"Task not found"}
```

شرح الصورة 42: توضح هذه الصورة استجابة النظام عند محاولة الوصول إلى مورد غير موجود، حيث يُعيد رمز الحالة 404 Not Found مع رسالة خطأ واضحة. ("Task not found") تُثبت هذه الصورة أن معالجة الأخطاء (Error Handling) مُطبَّقة بشكل صحيح في جميع عمليات الـ CRUD.

12. مساهمات الفريق (Team Contributions)

12.1 توزيع المهام

يُلخِّص الجدول التالي المسؤوليات والمهام التي قام بها كل عضو في فريق العمل:

المهام التفصيلية	الدور / المسؤولية	العضو
تطوير REST API باستخدام FastAPI، دمج PostgreSQL عبر psycopg، تنفيذ عمليات CRUD الكاملة، تصميم نماذج البيانات (Pydantic) (Models)، كتابة init.sql	مطور الـ Backend والـ API	أسامة الجهوري

المهام التفصيلية	الدور / المسؤولية	العضو
مراجعة وشرح بنية Dockerfile، تحليل Image Layers واستراتيجية Dependency Caching، التحقق من حجم الصورة النهائية	مراجع Dockerfile وتحسين الصور	عبدالله اليافعي
مراجعة وشرح تكوين Nginx Reverse Proxy، تحليل Network Architecture وعزل المنافذ، اختبار توجيه الطلبات	مراجع Nginx ومعمارية الشبكة	معتز ابركر
مراجعة وشرح Docker Compose والـ Persistence، إدارة Registry و Versioning، جمع أدلة الاختبار (Testing Evidence)	مراجع Docker وCompose والاختبار	عمر الاسيدي

12.2 ملاحظة

يجب أن يكون جميع أعضاء الفريق قادرين على شرح المعمارية الكاملة للنظام وأي جزء من المشروع أثناء التقييم الشفهي (Oral Assessment)، بغض النظر عن المسؤولية الأساسية المُسندة لكل عضو.

المراجع

- [Docker Documentation](#)
- [FastAPI Documentation](#)
- [PostgreSQL Documentation](#)
- [Nginx Documentation](#)
- [Docker Compose Specification](#)
- [Pydantic Documentation](#)